VARDHAMAN
COLLEGE OF ENGINEERING

A Case-Study Report on

TransitFlow:Optimizing Urban Traffic Signal Timing Using Reinforcement Signals

A9513 - Python Programming Laboratory

Submitted by

Batch No.

M.Siri Chandana	25881A0555
B.Sanjana	25881A0559
N.Yokshitha Sai	25881A0564

Course Facilitator

Dr.V.Lokeshwari Vinya

Assosiate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Contents

1 Introduction 2
1.1 Importance and Relevance to Engineering and Society 2
1.2 Objectives of the Case Study Analysis 2
2 Case Study Background 3
2.1 Description of the Case Scenario 3
2.2 Key Facts and Stakeholders Involved 3
2.3 Background Information 4
3 Problem Identification and Analysis 5
3.1 Main Challenge: Mismatch Between Fixed Timing and Live Demand . . . 5
3.2 Engineering Principles and Theories Applied 5
3.3 Analysis of the Identified Problem: Simulation Study 7
4 Application of Program Outcomes 10
5 Sustainable Development Goals (SDGs) Related to TransitFlow 11
5.1 SDG 9: Industry, Innovation and Infrastructure 11
5.2 SDG 11: Sustainable Cities and Communities 11
5.3 SDG 13: Climate Action 11
5.4 SDG 3: Good Health and Well-being 11
6 Proposed Solution and Recommendations 12
6.1 Proposed Solution 12
6.2 Recommendations 12
7 Conclusion and Lessons Learned 13
7.1 Conclusion 13
7.2 Lessons Learned 13
A Simulation Code and Output 14

1 Introduction

Urban road networks are controlled by traffic signals whose timing decides how long each stream of vehicles waits at an intersection. In many cities these signals still run on **fixed-time plans**: a cycle length and green splits computed offline from historical counts and then repeated all day. Real demand is not fixed. It rises and falls with the morning and evening peaks, changes direction (tidal flow), and varies from day to day. When a fixed plan is mismatched with the demand actually present, one approach is starved while another discharges into empty road, queues spill back, and **gridlock** can form at busy metropolitan intersections.

TransitFlow is the case study we use to examine an alternative: a signal controller that observes live vehicle density and *learns* when to change the light. The controller is framed as a reinforcement-learning agent: it senses the queue state, chooses to keep or switch the phase, and receives a "reinforcement signal" (a reward) that is higher when vehicles wait less. The study is built around three core concepts: **graph representation** of the road network, **state-space modeling** of the intersection, and **heuristic optimization** of signal timing.

1.1 Importance and Relevance to Engineering and Society

From an engineering perspective, signal timing is a sequential decision problem under uncertainty. It combines queueing theory, stochastic modeling, search and optimization, and machine learning, with real-time sensing as the data source. Classical solutions such as Webster's formula [1] and adaptive systems such as SCOOT [2] and SCATS [3] show that responsiveness to measured traffic pays off; reinforcement learning (RL) offers a way to obtain a responsive policy from experience rather than hand-written rules [4, 7, 8].

From a societal perspective, time lost in queues at intersections is time lost by commuters, buses, ambulances and goods vehicles, and idling engines add fuel use and local emissions. Better signal timing is attractive because it improves an existing asset (the signal controller) instead of requiring new road space.

1.2 Objectives of the Case Study Analysis

- To explain why fixed-time signal plans perform poorly under time-varying demand.
- To formulate signal control with a graph model of the network and a state-space (Markov decision process) model of one intersection.
- To implement in Python and compare three controllers on the same simulated traffic: fixed-time, a rule-based actuated heuristic, and a tabular Q-learning agent.
- To report the results honestly, including the limits of a simplified simulation.
- To map the work to Program Outcomes and Sustainable Development Goals and give recommendations for real deployment.

Scope and methodology. The study covers a single signalised four-way intersection with two conflicting phases. The methodology has five steps: (i) formulate the problem as a Markov decision process; (ii) build a seeded Python simulator of queues, arrivals and discharge; (iii) implement three controllers (fixed-time, actuated heuristic, Q-learning); (iv) train the learner and test all controllers on identical unseen traffic; and (v) analyse delay, queue length and robustness to demand changes.

Organization of the report. Section 2 gives the background and stakeholders. Section 3 identifies the problem, develops the models and presents the simulation results. Section 4 maps the work to Program Outcomes and Section 5 to Sustainable Development Goals. Section 6 proposes a solution with recommendations and an implementation roadmap, and Section 7

concludes with lessons learned and future scope. The Python code and its printed output are in Appendix A.

2 Case Study Background

2.1 Description of the Case Scenario

We consider a busy four-way intersection in a metropolitan area, controlled by a two-phase signal: phase NS gives green to north–south traffic and phase EW to east–west traffic. Between phases there is a 3 s yellow/all-red interval in which nothing discharges. Over one hour the demand pattern changes: it starts light and balanced, becomes heavy in the north–south direction, then heavy in the east–west direction, and finally returns to a moderate balanced level. This tidal pattern is a typical stress test, because any single fixed split is wrong for most of the hour.

The question TransitFlow asks is: *can a controller that sees the current queues decide better than a timer, and can it learn to do so without being given the rule?*

2.2 Key Facts and Stakeholders Involved

Key technical facts used in this study

- A lane discharges at a *saturation flow*; we use 1800 vehicles per hour (0.5 veh/s) per green approach, a common textbook order of magnitude.
- Every phase change wastes lost time (yellow/all-red). Switching too often wastes capacity; switching too rarely makes the other approach wait.
- Fixed-time control repeats one plan, whereas adaptive control (e.g. SCOOT, SCATS) adjusts timings from detector data [2, 3].

Stakeholders

- **City traffic authorities:** own the signals and set performance targets.
- **Commuters, public transport and emergency vehicles:** experience the delay directly.
- **Technology providers:** supply detectors, controllers and software.
- **Residents and environmental regulators:** affected by congestion-related noise and emissions.
- **Engineers and researchers:** responsible for safe, validated algorithms.

Table 1: Stakeholders and their main interests

Stakeholder	Main interest	Role in the system
City traffic authority	Lower delay and fewer complaints at low cost	Owns, operates and approves signal plans
Commuters and bus users	Short, predictable journey times	Source of demand; experience the outcome
Emergency services	Clear, fast routes	Need priority handling
Technology provider	Reliable, certifiable products	Supplies detectors and controllers
Regulators and residents	Lower noise and emissions	Set limits; affected by idling

2.3 Background Information

Signal timing has evolved in three stages. **Fixed-time** plans, e.g. Webster's method [1], choose a cycle length from measured flows. **Actuated and adaptive** systems use detectors to extend or terminate green and to adjust splits and offsets online [2, 3]. More recently, **reinforcement learning** has been applied, from tabular multi-agent methods [7] to deep RL using camera or sensor features [9, 10], and experimental reviews compare algorithms for this task [8]. Deep Q-networks [6] made large state spaces tractable; open simulators such as SUMO [11] are the standard way to train and test such agents before any field use.

Graph view of a city. A road network is naturally a directed graph $G = (V, E)$: each intersection is a vertex and each road segment a directed edge carrying attributes such as length and queue (Fig. 1). Graph representation lets one define neighbours, shortest paths (e.g. with A* and other heuristic search [12]) and, in the multi-intersection case, how congestion at one node propagates to adjacent nodes. Our experiments control a single node; the graph view explains how the solution would extend (Section 6).

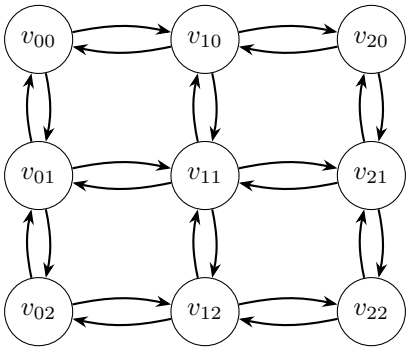


Figure 1: A 3×3 city grid as a directed graph: vertices are signalised intersections, edges are road links with queue length as an attribute.

2.3.1 Evolution of Signal Control Strategies

Table 2 compares the main families of signal control. The entries are qualitative and describe typical characteristics, not measured results.

Table 2: Comparison of traffic signal control strategies

Strategy	**Information used**	**Adapts to live traffic?**	**Typical limitation**
Fixed-time (Webster)	Historical flow counts	No	Wrong when demand differs from the plan
Actuated	Detector calls and gaps	Locally, within preset limits	Rules are hand-tuned per site
Adaptive (SCOOT, SCATS)	Network-wide detector data	Yes, by adjusting splits and offsets	Needs dense detectors and calibration
Reinforcement learning	State features and reward	Yes, policy learned from experience	Needs a simulator, training and validation

2.3.2 Why Intersections Become Gridlocked

- **Demand imbalance:** one approach receives most of the traffic while the timer gives equal green to both.
- **Lost time:** every phase change wastes seconds of yellow and start-up that no vehicle can use.
- **Spill-back:** a queue longer than its road link blocks the upstream intersection, which then cannot discharge either.
- **Randomness:** vehicle arrivals are random, so average-based plans cannot absorb short bursts.

3 Problem Identification and Analysis

3.1 Main Challenge: Mismatch Between Fixed Timing and Live Demand

The central problem is that a timer cannot see traffic. If the north–south queue is long and the east–west queue is empty, a fixed plan still hands green to the empty approach when its timer expires. The consequences are longer queues, longer delay, and a greater chance that a queue reaches the upstream intersection and blocks it.

3.2 Engineering Principles and Theories Applied

3.2.1 State-Space Modeling (Markov Decision Process)

We model one intersection as a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- **State** $s = (\text{phase}, b_{NS}, b_{EW}, m)$, where $b_{NS}, b_{EW} \in \{0, \dots, 6\}$ are binned queue lengths (edges $0, 1, 3, 6, 10, 15, 25$) and $m \in \{0, 1\}$ indicates whether the minimum green of 10 s has elapsed. This gives $2 \times 7 \times 7 \times 2 = 196$ states.
- **Action** $a \in \{\text{KEEP}, \text{SWITCH}\}$, chosen every $\Delta t = 5\,\text{s}$. A SWITCH inserts 3 s of yellow and is ignored while $m = 0$.
- **Reward** $r = -\frac{1}{100} \sum_{5\text{ s}} (q_{NS} + q_{EW})$, the negative queue-seconds; maximising return minimises waiting.
- **Transition** P is stochastic: Bernoulli arrivals with time-varying rate and random discharge at saturation flow.

Table 3: MDP formulation of the intersection

Element	**Definition used in the code**
State	(phase, queue bin NS, queue bin EW, min-green flag); 196 states
Action	KEEP the current phase or SWITCH (3 s yellow, then the other approach is green)
Reward	Negative total queue length summed over the 5 s decision step, divided by 100
Decision interval	5 s (minimum green 10 s, maximum green 60 s for the heuristic)
Discount factor	$\gamma = 0.95$

3.2.2 Q-Learning

The agent learns an action-value function with the Watkins update [5]

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big], \tag{1}$$

with $\alpha = 0.1$, $\gamma = 0.95$ and ϵ-greedy exploration decaying linearly from 1.0 to 0.05 over the first 70% of 400 training episodes. The learned policy is $\pi(s) = \arg\max_a Q(s,a)$. Figure 2 shows the loop.

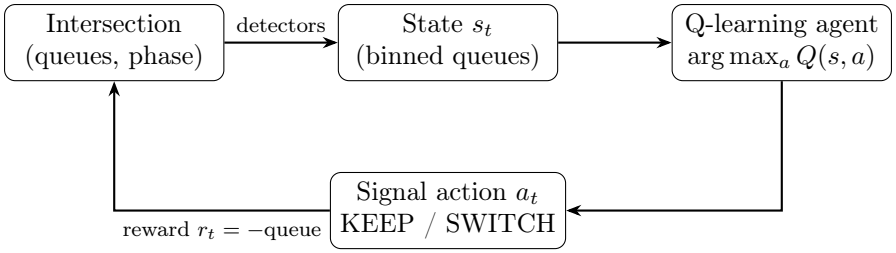


Figure 2: Reinforcement-learning loop for signal control.

The training procedure implemented in the code is:

1. Start a new episode (one simulated hour) with a random seed and a random demand scale between 0.8 and 1.2.
2. Observe the binned state s. With probability ϵ pick a random action, otherwise pick the action with the largest $Q(s,a)$.
3. Apply the action for 5 s of simulated traffic and observe the reward r and the next state s'.
4. Update $Q(s,a)$ with equation (1).
5. Repeat until the episode ends; decay ϵ; after 400 episodes freeze the table and always act greedily.

3.2.3 Performance Metrics

- **Average queue (veh):** total queue-seconds divided by episode length.
- **Delay per vehicle (s):** total queue-seconds divided by the number of vehicles that arrived (Little's law).
- **Vehicles served:** departures during the hour (throughput).
- **Queue at end (veh):** vehicles still waiting at 3600 s, which shows whether a controller left a backlog.

3.2.4 Heuristic Optimization and the Baselines

- **Fixed-time:** switch when the current green has lasted 30 s. Webster's formula [1], $C_0 = (1.5L+5)/(1-Y)$, with total lost time $L = 6$ s and peak critical-flow ratio $Y = 0.22/0.5 + 0.10/0.5 = 0.64$, gives $C_0 \approx 38.9$ s, so a 30 s+30 s plan is a reasonable timer for this demand.
- **Actuated heuristic:** switch when the red approach has at least 3 more waiting vehicles than the green one (or the green approach is empty and the other is not), never before 10 s of green and always after 60 s. This is a hand-designed rule that uses the same live queue information as the learner.
- **Q-learning:** the same information, but the rule is learned from reward.

3.3 Analysis of the Identified Problem: Simulation Study

3.3.1 Experimental Setup

All code is in Appendix A and runs with Python 3 and NumPy only. The simulator advances in 1 s steps over a 3600 s episode. Training episodes use fresh random seeds and a random demand scale from 0.8 to 1.2. Testing uses 30 *unseen* seeds (100–129). Arrivals and service randomness use separate random streams, so all controllers face exactly the same arriving vehicles on each seed. Delay per vehicle is computed as total queue-seconds divided by vehicles arrived (an application of Little's law). The settings are listed in Table 4.

Table 4: Simulation parameters (as coded)

Parameter	**Value**
Time step / episode length	1 s / 3600 s
Saturation flow of a green approach	0.5 veh/s (1800 veh/h)
Yellow / all-red lost time	3 s per switch
Decision interval / minimum green	5 s / 10 s
Demand 0–900 s (NS, EW)	0.10, 0.10 veh/s
Demand 900–1800 s (NS, EW)	0.22, 0.10 veh/s
Demand 1800–2700 s (NS, EW)	0.10, 0.22 veh/s
Demand 2700–3600 s (NS, EW)	0.15, 0.15 veh/s
Q-learning: α, γ, episodes	0.1, 0.95, 400
Test seeds	100 to 129 (30 unseen runs)

3.3.2 Results

Table 5: Test results at nominal demand: mean ± standard deviation over 30 unseen seeds.

Controller	Avg. queue (veh)	Delay (s/veh)	Vehicles served	Queue at end
Fixed-time (30 s)	8.05 ± 1.96	28.1 ± 6.4	1024	5.3
Actuated heuristic	3.23 ± 0.26	11.3 ± 0.8	1026	3.0
Q-learning (greedy)	3.25 ± 0.24	11.4 ± 0.7	1026	3.0

Both queue-aware controllers reduce mean delay by about 60% relative to the fixed-time plan (actuated: −59.8%, Q-learning: −59.5%). The two are statistically indistinguishable at nominal demand: their mean delays differ by 0.1 s/veh while the standard deviations are 0.7–0.8 s/veh.

Observations. (a) The three controllers serve almost the same number of vehicles (1024 to 1026), so the difference is not throughput but waiting time and backlog. (b) The fixed-time plan leaves a larger queue at the end of the hour (5.3 vehicles against 3.0). (c) The variation between runs is much larger for fixed-time (standard deviation 6.4 s/veh) than for the queue-aware controllers (0.7 to 0.8 s/veh), i.e. its service is also less predictable.

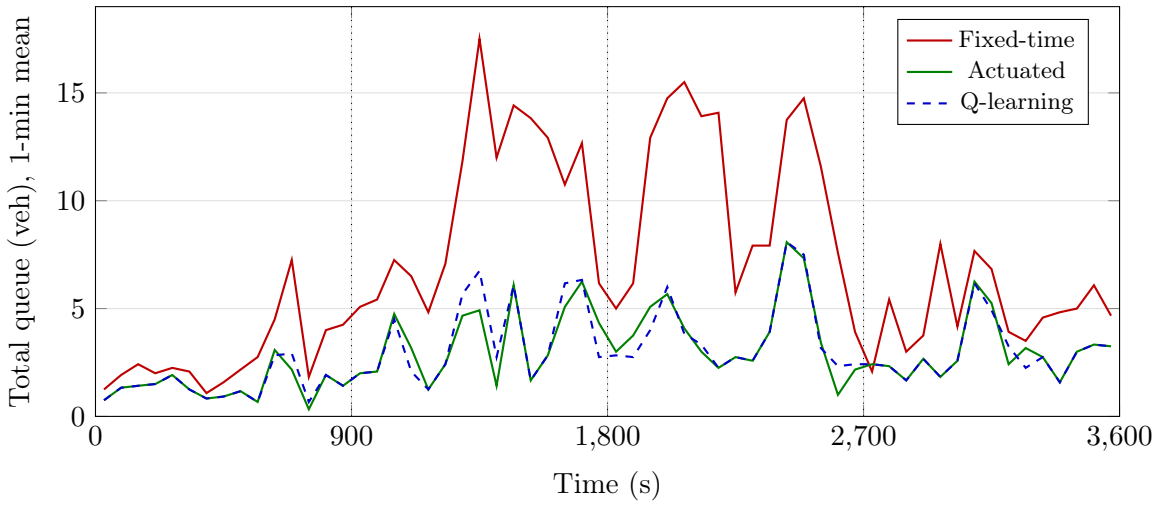


Figure 3: Total queue over time for test seed 100 (mean of each 60 s window). Dotted lines mark demand changes. The fixed-time plan builds large queues when demand becomes lopsided (900–1800 s and 1800–2700 s).

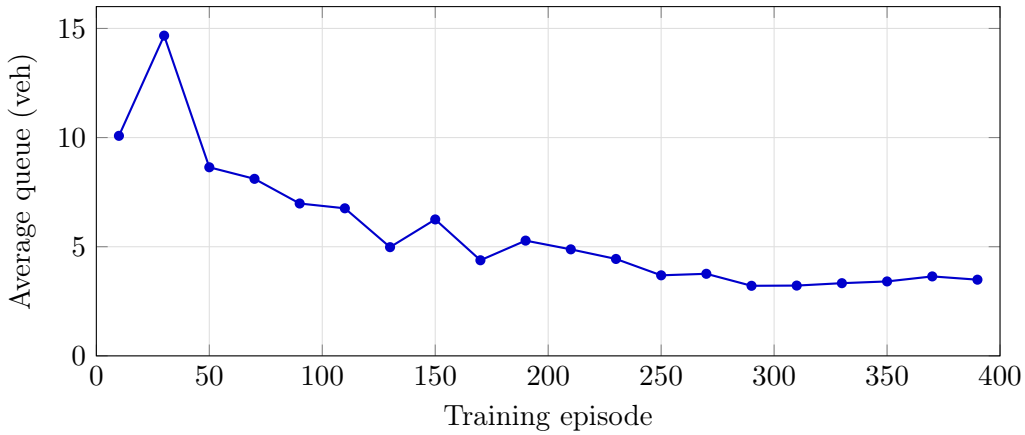


Figure 4: Training curve: average queue, averaged over blocks of 20 training episodes, falls and stabilises after roughly 250 episodes as exploration decays.

3.3.3 Robustness to Demand Level

Table 6: Mean delay (s/veh) over 30 test seeds when all demand rates are scaled.

Controller	×0.8	×1.0	×1.2
Fixed-time (30 s)	18.0	28.1	60.7
Actuated heuristic	9.2	11.3	16.5
Q-learning (greedy)	9.3	11.4	15.6

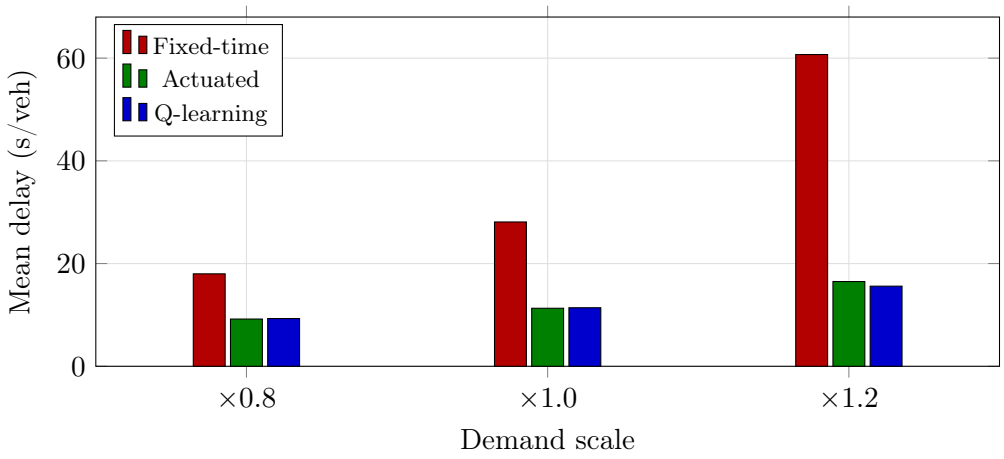


Figure 5: Delay versus demand scale. The fixed-time plan degrades sharply near capacity.

3.3.4 Interpretation and Limitations

- **What the data support:** responding to live queues cuts delay substantially compared with a timer, and a tabular Q-learning agent can discover such a policy from reward alone, without being told the rule.
- **What they do not show:** Q-learning did *not* beat the simple actuated heuristic at nominal demand. At ×1.2 demand it was lower by 0.9 s/veh (15.6 vs. 16.5), but we did not test whether that gap is statistically significant, so we do not claim an advantage.

- **Simplifications:** one isolated intersection, two phases, no turning movements, pedestrians, buses or incidents; Bernoulli arrivals; perfect and noise-free queue sensing; no travel time between intersections. Absolute numbers apply to this model only and must not be read as predictions for a real city. Field results would need validation in a calibrated simulator such as SUMO [11] first.

4 Application of Program Outcomes

The case study on **TransitFlow: Optimizing Urban Traffic Signal Timing Using Reinforcement Signals** applies the following Program Outcomes (POs).

Table 7: Mapping of Program Outcomes to the Case Study

PO No.	Program Outcome and Relevance to the Case Study
PO1	**Engineering Knowledge:** Applies probability, queueing, graph theory and reinforcement learning to model and control an intersection.
PO2	**Problem Analysis:** Identifies the mismatch between fixed timing and time-varying demand and formulates it as an MDP.
PO3	**Design/Development of Solutions:** Designs the state, action and reward and implements a Q-learning controller with safety constraints (minimum/maximum green, yellow time).
PO4	**Conduct Investigations of Complex Problems:** Runs controlled experiments on 30 unseen seeds and varied demand levels and reports variability and limits.
PO5	**Modern Tool Usage:** Uses Python/NumPy simulation and plotting tools; identifies SUMO as the next validation tool.
PO6	**The Engineer and The World:** Considers commuter time, emergency access and urban emissions.
PO7	**Ethics:** Reports that Q-learning did not outperform the heuristic at nominal demand, and avoids presenting simulation numbers as real-world claims; discusses safety and fairness in deployment.
PO8	**Individual and Collaborative Teamwork:** The work was divided among three team members across modeling, coding and reporting.
PO9	**Communication:** Presents methods, results and limits in a structured written report with figures and tables.
PO10	**Project Management and Finance:** Recommends staged piloting from simulation to a single junction to a corridor, and compares the cost of retrofitting controllers against building new road capacity.
PO11	**Life-Long Learning:** Motivates continued study of deep and multi-agent RL and of traffic engineering standards.

5 Sustainable Development Goals (SDGs) Related to TransitFlow

The deployment of TransitFlow-style adaptive signal control aligns with several United Nations **Sustainable Development Goals (SDGs)** by improving urban transport, making better use of existing infrastructure and potentially reducing congestion-related emissions. Table 8 summarises the goals and the sections below give the details.

Table 8: SDGs addressed by TransitFlow

SDG	Target	Contribution of the case study
SDG 9	9.1, 9.5	Smarter use of existing signal infrastructure; applied AI research
SDG 11	11.2, 11.6	More reliable road transport; possible reduction of idling emissions (not measured here)
SDG 13	13.2	Low-cost traffic management measure for urban climate plans (savings not quantified)
SDG 3	3.9	Possible reduction in exposure to traffic-related air pollution

5.1 SDG 9: Industry, Innovation and Infrastructure

Target 9.1: Develop quality, reliable, sustainable and resilient infrastructure.
Target 9.5: Enhance scientific research and upgrade technological capabilities.

- Upgrades existing signal infrastructure with data-driven control instead of new construction.
- Advances applied AI for transport systems.

5.2 SDG 11: Sustainable Cities and Communities

Target 11.2: Provide access to safe, affordable, accessible and sustainable transport systems for all.
Target 11.6: Reduce the adverse per-capita environmental impact of cities, including air quality.

- Lower intersection delay improves the reliability of road and bus travel.
- Less stop-and-go idling *may* reduce local emissions; this was not measured in our simulation.

5.3 SDG 13: Climate Action

Target 13.2: Integrate climate change measures into policies, strategies and planning.

- Efficient traffic management is one low-cost measure that can be part of urban climate plans. Quantifying fuel and CO_2 savings would require an emissions model, which is future work.

5.4 SDG 3: Good Health and Well-being

Target 3.9: Substantially reduce illnesses from air pollution.

- Shorter queues can reduce residents' exposure to traffic-related pollution; priority for emergency vehicles is a natural extension.

6 Proposed Solution and Recommendations

6.1 Proposed Solution

- **Queue-aware control:** deploy detector-driven control at each signalised intersection, using either an actuated rule or a learned policy with the same inputs.
- **Safety layer:** keep hard constraints outside the learner: minimum green, maximum green, yellow/all-red and pedestrian phases.
- **Train in simulation first:** calibrate a SUMO model [11] with local counts, train and test there, then pilot at one junction.
- **Scale through the graph:** treat the network as a graph $G = (V, E)$; each node runs a local agent that also observes the queues on neighbouring edges, following multi-agent approaches [7, 10].
- **Fallback:** on detector or software failure, revert automatically to a fixed-time plan.

6.2 Recommendations

1. Begin with the simple actuated heuristic as the benchmark; adopt a learned controller only where it is shown to beat it in calibrated tests.
2. Validate with realistic arrivals, turning movements, pedestrians and buses before field use.
3. Use function approximation (e.g. deep Q-networks [6]) when the state includes more lanes or neighbouring intersections; the tabular method does not scale.
4. Report variability across many random seeds, not a single run.
5. Monitor sensors and log decisions so that engineers can audit the controller.
6. Add priority for emergency and public transport vehicles.
7. Pilot in stages: one junction, a corridor, then a district.

6.2.1 Implementation Roadmap

Table 9: Staged deployment plan

Phase	Activity	Exit criterion
1	Calibrate a simulator with local traffic counts	Simulated flows match field counts
2	Train and compare fixed-time, actuated and learned controllers	Learned policy is better than the best baseline over many seeds
3	Shadow mode at one junction (decisions logged, not applied)	Logged decisions are safe and sensible
4	Live pilot at one junction with fallback to fixed-time	Measured delay improves; no safety incidents
5	Extend to a corridor, then a district, using the graph model	Benefits hold with neighbouring junctions interacting

6.2.2 Risks and Mitigation

Table 10: Risks and mitigation measures

Risk	**Why it matters**	**Mitigation**
Policy fails outside training conditions	Real traffic differs from the simulator	Train on varied demand; keep rule-based fallback
Detector or software fault	Wrong state leads to wrong decisions	Sensor health checks; automatic fixed-time mode
Unsafe timing	A learner could shorten green or yellow	Hard limits for min/max green and yellow outside the learner
Unfairness to minor roads	Reward may favour the busy approach	Add waiting-time or fairness terms to the reward
Lack of transparency	Operators must trust and audit it	Log states, actions and rewards for review

7 Conclusion and Lessons Learned

7.1 Conclusion

This case study examined whether live queue information can improve signal timing. In a simulated intersection with changing demand, controllers that react to queues reduced mean delay by about 60% relative to a 30 s fixed-time plan, and the advantage grew as demand approached capacity. A tabular Q-learning agent learned an effective policy from reward alone and matched, but did not clearly beat, a well-designed actuated heuristic. The graph, state-space and heuristic-optimization concepts together give a clear path from a single junction to a city network, with validation in calibrated simulation as the essential next step.

7.2 Lessons Learned

1. **Fixed timing is brittle:** its delay rose more than three-fold between 0.8× and 1.2× demand (18.0 to 60.7 s/veh).
2. **A strong baseline matters:** the simple heuristic equalled the learner; without it we could have overstated RL.
3. **Reward design drives behaviour:** negative queue-seconds is simple and effective, but real systems need terms for fairness, pedestrians and emissions.
4. **Evaluate fairly:** unseen seeds and identical arrivals for every controller are needed for a trustworthy comparison.
5. **Simulation is not reality:** results here support the approach but do not predict field performance.

Future scope:

- Extend the simulator to a grid of intersections using the graph model, with vehicles travelling between nodes.
- Add turning movements, pedestrian phases and bus or emergency-vehicle priority.

- Replace the table with a neural network (deep Q-network) and compare against the heuristic under statistical tests.
- Validate in SUMO with calibrated demand, and add emission and fuel-use measures to the reward.
- Study noisy or missing sensor data and the behaviour of the controller under detector failure.

A Simulation Code and Output

The listing below is the exact file `transitflow_sim.py` used to produce every number in Section 3 (run with `python3 transitflow_sim.py`; all randomness is seeded, so the output is reproducible). Save it as a Python file to re-run it.

```
import numpy as np

EP_LEN, DT, YELLOW, MIN_GREEN, MAX_GREEN, SAT = 3600, 5, 3, 10, 60, 0.5
SCHEDULE = [(0, .10, .10), (900, .22, .10), (1800, .10, .22), (2700, .15, .15)]
QB = [0, 1, 3, 6, 10, 15, 25]
N_BINS = len(QB)


def demand(t, scale=1.0):
    row = [r for r in SCHEDULE if t >= r[0]][-1]
    return row[1] * scale, row[2] * scale


def qbin(q):
    return int(np.searchsorted(QB, q, side="right") - 1)


class Intersection:
    def __init__(self, seed, scale=1.0):
        self.rng = np.random.default_rng(seed)
        self.srv = np.random.default_rng(seed + 1_000_000)
        self.scale = scale
        self.t, self.q, self.phase, self.green_t = 0, [0.0, 0.0], 0, MIN_GREEN
        self.arrived, self.served, self.queue_seconds = 0, 0, 0.0

    def state(self):
        return (self.phase, qbin(self.q[0]), qbin(self.q[1]), int(self.green_t >= MIN_GREEN))

    def step(self, action):
        if action == 1 and self.green_t < MIN_GREEN:
            action = 0
        cost = 0.0
        for k in range(DT):
            lam = demand(self.t, self.scale)
            for d in (0, 1):
                if self.rng.random() < lam[d]:
                    self.q[d] += 1
                    self.arrived += 1
            in_yellow = action == 1 and k < YELLOW
            if action == 1 and k == YELLOW:
                self.phase, self.green_t = 1 - self.phase, 0
            if not in_yellow and self.q[self.phase] > 0 and self.srv.random() < SAT:
                self.q[self.phase] -= 1
                self.served += 1
            cost += self.q[0] + self.q[1]
            self.t += 1
            if not in_yellow:
                self.green_t += 1
        self.queue_seconds += cost
        return -cost / 100.0, self.t >= EP_LEN


def fixed_time(env, green=30):
```

```
    return int(env.green_t >= green)


def actuated(env):
    cur, oth = env.q[env.phase], env.q[1 - env.phase]
    if env.green_t >= MAX_GREEN:
        return 1
    return int(env.green_t >= MIN_GREEN and (oth - cur >= 3 or (cur == 0 and oth > 0)))


def run_episode(policy, seed, scale=1.0, trace=False):
    env = Intersection(seed, scale)
    qs = []
    done = False
    while not done:
        a = policy(env)
        _, done = env.step(a)
        if trace:
            qs.append(env.q[0] + env.q[1])
    out = dict(avg_queue=env.queue_seconds / EP_LEN,
               delay=env.queue_seconds / max(env.arrived, 1),
               arrived=env.arrived, served=env.served, final_q=env.q[0] + env.q[1])
    return (out, qs) if trace else out


def train_q(episodes=400, alpha=0.1, gamma=0.95, eps0=1.0, eps_min=0.05, seed=0):
    rng = np.random.default_rng(seed)
    Q = np.zeros((2, N_BINS, N_BINS, 2, 2))
    curve = []
    for ep in range(episodes):
        eps = max(eps_min, eps0 * (1 - ep / (0.7 * episodes)))
        env = Intersection(seed=10_000 + ep, scale=rng.uniform(0.8, 1.2))
        s, done, total = env.state(), False, 0.0
        while not done:
            a = int(rng.integers(2)) if rng.random() < eps else int(np.argmax(Q[s]))
            r, done = env.step(a)
            s2 = env.state()
            target = r + (0 if done else gamma * np.max(Q[s2]))
            Q[s][a] += alpha * (target - Q[s][a])
            s, total = s2, total + r
        curve.append(env.queue_seconds / EP_LEN)
    return Q, curve


def make_greedy(Q):
    def policy(env):
        s = env.state()
        return int(np.argmax(Q[s]))
    return policy


if __name__ == "__main__":
    import json
    Q, curve = train_q()
    policies = {"Fixed-time (30 s)": fixed_time, "Actuated heuristic": actuated,
                "Q-learning (greedy)": make_greedy(Q)}
    seeds = range(100, 130)
    results = {}
    print(f"{'Controller':<22}{'Avg queue (veh)':>18}{'Delay (s/veh)':>16}{'Served':>10}{'Final queue
    ':>14}")
    for name, pol in policies.items():
        runs = [run_episode(pol, s) for s in seeds]
        m = {k: (float(np.mean([r[k] for r in runs])), float(np.std([r[k] for r in runs])))
             for k in runs[0]}
        results[name] = m
        print(f"{name:<22}{m['avg_queue'][0]:>12.2f} +/-{m['avg_queue'][1]:>4.2f}"
              f"{m['delay'][0]:>10.1f} +/-{m['delay'][1]:>4.1f}{m['served'][0]:>10.0f}{m['final_q
    '][0]:>14.1f}")
    base = results["Fixed-time (30 s)"]["delay"][0]
    for name, m in results.items():
        print(f"{name:<22} delay change vs fixed-time: {100*(m['delay'][0]-base)/base:+.1f}%")
    print("\nRobustness (mean delay s/veh over 30 seeds):")
    rob = {}
```

```
    for sc in (0.8, 1.0, 1.2):
        row = {n: float(np.mean([run_episode(p, s, sc)["delay"] for s in seeds])) for n, p in policies.
     items()}
        rob[sc] = row
        print(f"  demand x{sc}: " + ", ".join(f"{n}={v:.1f}" for n, v in row.items()))
    np.save("q_table.npy", Q)
    json.dump({"curve": curve, "results": results, "robust": {str(k): v for k, v in rob.items()}},
              open("results.json", "w"))
```

Program output

```
PS C:\Users\Dell> cd "C:\Users\Dell\OneDrive\Desktop"
PS C:\Users\Dell\OneDrive\Desktop> py transitflow_sim.py
Controller              Avg queue (veh)   Delay (s/veh)    Served   Final queue
Fixed-time (30 s)            8.05 +/-1.96      28.1 +/- 6.4      1024           5.3
Actuated heuristic           3.23 +/-0.26      11.3 +/- 0.8      1026           3.0
Q-learning (greedy)          3.25 +/-0.24      11.4 +/- 0.7      1026           3.0
Fixed-time (30 s)      delay change vs fixed-time: +0.0%
Actuated heuristic     delay change vs fixed-time: -59.8%
Q-learning (greedy)    delay change vs fixed-time: -59.5%

Robustness (mean delay s/veh over 30 seeds):
  demand x0.8: Fixed-time (30 s)=18.0, Actuated heuristic=9.2, Q-learning (greedy)=9.3
  demand x1.0: Fixed-time (30 s)=28.1, Actuated heuristic=11.3, Q-learning (greedy)=11.4
  demand x1.2: Fixed-time (30 s)=60.7, Actuated heuristic=16.5, Q-learning (greedy)=15.6
PS C:\Users\Dell\OneDrive\Desktop>
```

Figure 6: Output of `transitflow_sim.py` in the VS Code terminal.

Bibliography

[1] F. V. Webster, "Traffic signal settings," Road Research Technical Paper No. 39, Her Majesty's Stationery Office, London, 1958.

[2] P. B. Hunt, D. I. Robertson, R. D. Bretherton and R. I. Winton, "SCOOT – a traffic responsive method of coordinating signals," TRRL Laboratory Report LR 1014, Transport and Road Research Laboratory, Crowthorne, UK, 1981.

[3] P. R. Lowrie, "The Sydney coordinated adaptive traffic system – principles, methodology, algorithms," in *Proc. IEE Int. Conf. Road Traffic Signalling*, London, 1982.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[5] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[6] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[7] M. Wiering, "Multi-agent reinforcement learning for traffic light control," in *Proc. 17th Int. Conf. Machine Learning (ICML)*, 2000, pp. 1151–1158.

[8] P. Mannion, J. Duggan and E. Howley, "An experimental review of reinforcement learning algorithms for adaptive traffic signal control," in *Autonomic Road Transport Support Systems*, Springer, 2016, pp. 47–66.

[9] H. Wei, G. Zheng, H. Yao and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. ACM SIGKDD*, 2018, pp. 2496–2505.

[10] T. Chu, J. Wang, L. Codecà and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Trans. Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.

[11] P. A. Lopez *et al.*, "Microscopic traffic simulation using SUMO," in *Proc. IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575–2582.

[12] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.